NOCK & THE ART OF SOLID-STATE COMPUTING
An Enquiry into the Virtual

N. E. Davis
~lagrev-nocfep

Contents

Chapter 1

The Roots of Computing in the Ancient World

Abstract

Cuneiform and early computational concepts are explored in this chapter.

> Machines are pure products of human intellect, constructed for man's own purposes. They are nowhere to be found in the world of nature (as products of nature); yet the workings of the laws of nature find their purest expression in machines, purer than in any of the products of nature itself. The laws of nature work directly in machines, with an immediacy not to be found in the products of nature.
>
> (Keiji Nishitani, *Religion and Nothingness*, 1982, pp. 129–130)

There are many worthy raconteurs of the history of computing, such as **Hofstadter1980** (**Hofstadter1980**) and **Penrose2004** (**Penrose2004**). What distinguishes my approach to the history of computing from other tellings? Computing has deep connexions to alchemy and true language, which I hope to demonstrate without assuming any Hermetic background on the reader's part. Much like technology, computing is not merely a tool but a stance towards the world. However, it does not entail technologism's supercession; computing is always both a lens and a kaleidoscope, through which many layers of reality can be seen simultaneously. It is a way of seeing the world as a system of pure statements and acting to make those statements true. It is both ontology and epistemology.

First, therefore, I must establish that computation is the apex of a long human search for a language of truth. The origin of symbol itself in the σύμβολον, a

N. E. Davis ~lagrev-nocfep. "The Roots of Computing in the Ancient World". *The Nock Book* (2025): 1–x.

token broken in half to infallibly mark identity, is one font. Later "symbol" was the profession of faith, both an attempt to formulate truth and a sign of shared claims. The primary source, however, is language itself, Hofstadter's "strange loop" of self-reference. The first possible Nock formula is a crash, the void. The second is self-reference, awakening. Peirce would be proud of the progression. We have tricked ourselves into thinking that computing is a late development, a fruitful elaboration of the end of mathematics. In fact, it is the philosopher's stone, and it underlies every process in the world. It has turned lead into gold.

The historical account which follows aims throughout at the goal of understanding what Nock is and why it matters as a protocol. The reader will understand what mankind has been looking for, when we have caught various pieces of the puzzle, how these things came together in the mid-20th century, and how they have been refined into a pure machine.

Outline: - Cuneiform, tabulation, calculation, and algorithm - Solid-state indices: abaci, tokens, etc. Mechanism as embodied equation. - Pure language and true statements (Llull, Leibniz, Wilkins) - Generalization (Jacquard, Babbage) - The universal machine (Turing, Gödel, Zuse, von Neumann) - Pure statements as Pages from the Book (Curry, Church)

1.1 Tabulation

1.2 Solid-state indices

The problem is that any calculation lives in someone's head or someone's claim. This is a summary, but not the proof or solution.

Chapter 2

Combinators for Fun and Profit

Abstract

The first section of this chapter will introduce the basic `SKI` and `BCKW` combinator calculi and examine how they can represent each other. The second section will introduce Nock's opcodes as combinators, and show how they can be used to express various combinator idioms. The third section will show how to build practical affordances for common programming tasks, such as conditionals, recursion, subprocedure invocation, and data structures. The final section will discuss the relationship between combinators and virtualization, and how Nock can be used to build a virtual machine even in a crash-only context.

2.1 Combinators in Computing

The pioneers of computing elaborated three complementary visions:

1. The Turing machine, a simple abstract machine that can compute anything computable.
2. The lambda calculus, a pure functional language based on substitution and abstraction.
3. Combinatory logic, a variable-free functional language based on combinators.

Logicians early demonstrated the equivalence of these approaches, but particular formulations are still preferred for particular tasks. The Turing machine is the most intuitive model of computation, and is often used to prove computability results. The lambda calculus is the basis of many functional programming

languages, such as Lisp, Scheme, and Haskell. Combinatory logic is less widely used in practice, but has a long history in mathematical logic and type theory. It is also the basis of Nock.

Before we get too far out over our skis, let's talk about what a combinator is. At its simplest, a combinator is a function that takes one or more arguments and returns a result, without referring to any variables outside its own arguments. In practice, we'll use different combinators to build up more complex functions, and we'll see informally how they can be combined to express any computable function. We can even build combinators out of other combinators. The objective of combinatory logic is to express all computation in terms of combinators alone, without the need for variables or named functions.

This first section will introduce some elemental and compound combinators and make the case that they circumscribe "computing" as a concept.

2.1.1 The `SKI` calculus

Computable functions can be expressed using minimal sets of combinators. Many logicians today prefer to work with Moses Schönfinkel and Haskell Curry's `SKI` combinator calculus, which has three combinators:

The identity combinator `I` is conventionally written `Ix = x`. This simply means that it returns its single argument unchanged. There is little to say of this combinator at this point, but logicians have found that it frequently simplifies expressions.

```
I x
x
```

The constant combinator `K` is conventionally written `Kxy = x`. This means that it takes two arguments and returns the first, ignoring the second. It is useful for building functions that always return a fixed value.

Questions

1. What is the result of `I x`?
2. What is the result of `I I x`?

```
K x y
x
```

The substitution combinator `S` is conventionally written `Sxyz = xz(yz)`. This means that it takes three arguments: a function `x`, and two values `y` and `z`. It applies the function `x` to the value `z`, and also applies the function `y` to the value

z, then applies the result of `x z` to the result of `y z`. This combinator is more complex than the others, but it is very powerful, as it allows us to express function application and composition.

```
S x y z
x z (y z)
```

The SKI system is Turing complete (**blah**, **blah**), meaning that any computable function can be expressed using only these three combinators. It is also equivalent to the lambda calculus, meaning that any lambda expression can be translated into an equivalent SKI expression and vice versa. (cite church-turing thesis stuff)

Combinators can be combined to form more complex expressions. For example, the combinator `S K K` is equivalent to the identity combinator `I`:

```
S K K x
K x (K x)
x
```

2.1.2 The BCKW calculus

The BCKW combinator calculus is an alternative to SKI, with a different set of primitive combinators. It has four combinators:

The composition combinator `B` is conventionally written `Bxyz = x(yz)`. This means that it takes three arguments: a function `x`, and two values `y` and `z`. It applies the function `y` to the value `z`, then applies the function `x` to the result.

```
B x y z
x (y z)
```

The SKI and BCKW systems are equivalent, meaning that any expression in one system can be translated into an equivalent expression in the other system. For example, the identity combinator `I` can be expressed in BCKW as:

```
I = W K
```

```
    U
    Y
```

```
Y = S (K (S I I)) (S (S (K S) K) (K (S I I)))
```

MABT or whatever it is

Chris Barker The iota combinator ɩ can produce all the others, but in convoluted ways.

Combinator calculi are generally considered too minimalist for practical programming; they shine, however, in mathematical proofs. This means that we can use them to define a standard of behavior for an algorithm, but implement the

logic in a compliant form using whatever computational tools we like. Thus it is better to think of Nock not as a bare programming language, but as a standard of computation which can under many circumstances be used as the computation itself.[1]

2.2 Nock Opcodes as Combinator

The Nock specification is produced in full in Listing 2.1.

Listing 2.1: Nock 4K

```
Nock 4K

A noun is an atom or a cell. An atom is a natural number. A cell is
    an ordered pair of nouns.

Reduce by the first matching pattern; variables match any noun.

nock(a)             *a
[a b c]             [a [b c]]

?[a b]              0
?a                  1
+[a b]              +[a b]
+a                  1 + a
=[a a]              0
=[a b]              1

/[1 a]              a
/[2 a b]            a
/[3 a b]            b
/[(a + a) b]        /[2 /[a b]]
/[(a + a + 1) b]    /[3 /[a b]]
/a                  /a

#[1 a b]            a
#[(a + a) b c]      #[a [b /[(a + a + 1) c]] c]
#[(a + a + 1) b c]  #[a [/[(a + a) c] b] c]
#a                  #a

*[a [b c] d]        [*[a b c] *[a d]]

*[a 0 b]            /[b a]
```

[1] Consideration of when to do so is largely driven by performance bottlenecks.

```
*[a 1 b]              b
*[a 2 b c]            *[*[a b] *[a c]]
*[a 3 b]              ?*[a b]
*[a 4 b]              +*[a b]
*[a 5 b c]            =[*[a b] *[a c]]

*[a 6 b c d]          *[a *[[c d] 0 *[[2 3] 0 *[a 4 4 b]]]]
*[a 7 b c]            *[*[a b] c]
*[a 8 b c]            *[[*[a b] a] c]
*[a 9 b c]            *[*[a c] 2 [0 1] 0 b]
*[a 10 [b c] d]       #[b *[a c] *[a d]]

*[a 11 [b c] d]       *[[*[a c] *[a d]] 0 3]
*[a 11 b c]           *[a c]

*a                    *a
```

2.2.1 `I` Identity = Opcode 0

Nock's binary tree addressing is fully general and yields the identity combinator `I` as a special case. (The cost, of course, is dragging in some machinery for basic arithmetic, as we will see.)

Conceptually, we could write this in a hybrid form between combinators and Nock thus:

```
  *[a I b] == [0 1]
```

2.2.2 `K` Constant = Opcode 1

2.2.3 `S` Substitution = Opcode 2

2.2.4 The Axiomatic Operators

Because of Nock's cellular nature as a binary tree, we need the ability to access and manipulate nouns.

opcode 3, 4, 5, 10

2.3 Idioms & Design Patterns

Several of the combinators we examined above are not present in Nock *simpliciter*, but we can build them out of the other pieces. There are also common idioms

which are baked in as "compound opcodes" (such as conditional branching, opcode 6).

2.3.1 C Reversal

C = S (S (K (S (K S) K)) S) (K K)

In Nock, the reversal combinator `C` is almost free due to the binary tree nature of cells.

```
*[a C b] == *[7 [0 3] [[0 3] [0 2]]]
*[a C b] == *[[0 7] [0 6]]
```

Recursion we get for free with opcode 0.

μ-recursive functions

2.4 Virtualization

Nock is a crash-only pure language, which means at the base level it can only succeed and yield a new noun: no side effects, no exceptions, no state. Like a castaway on a desert island, we must build everything we need from raw materials at hand.

Nock has one trick built in and a couple of design patterns to help us get there:

1. Opcode 11 lets us send a hint to the runtime, which can be used to trigger side effects. The Nock expression formally doesn't know anything about these, but we can use them to print results, read input, and so forth.

2. The runtime can maintain state outside of Nock and supply it as a subject to subsequent Nock expressions. We will do a lot more with this later, but we want to highlight it as a key affordance now.

3. The virtualization pattern means that we can run Nock in Nock.

Opcode 11 can encode anything we want: the result of a hint is used by the runtime, and so anything the runtime is willing to work with can be produced.

The magic formula at the beating heart of a practical Nock operating system is simply:

```
*([state -.events] [9 2 10 [6 0 3] 0 2])
```

That is, apply the Nock formula against a subject pair consisting of the current state and the head of the list of events to process.

Chapter 3

Interpreter Runtime

Abstract

Now that we have established a firm foundation for idiomatic programming in Nock, we can build an interpreter using a higher-level language to execute Nock formulas. At this point, we will build the simplest possible Nock interpreter, a tree-walking interpreter. This requires a cursor and some basic memory management but not much more. We will implement some minimalist static hints to permit side effects like output.